

NutriScope — Cahier des TP

Un sujet par séance, du 29 juillet à la certification

Ihab Abadi — UTOPIOS · Cours Développeur en IA (RNCP 38603)

Promotion juillet 2026 – mars 2027 · Version 1 — 28 juillet 2026

Le principe

Ce projet n'est pas un exercice de plus. C'est **un seul et même produit** que vous allez construire pendant huit mois, en équipe, et que vous ferez grandir au rythme des modules : chaque séance « TP Projet fil rouge » du planning sert à appliquer immédiatement ce que vous venez de voir en cours.

Au total, le fil rouge représente **216 heures**, du 29 juillet à la préparation de la certification en mars. C'est votre principal terrain de preuve pour le jury : les cinq blocs de compétences du titre y trouvent chacun leur illustration concrète.

À la fin, vous aurez dans votre portfolio une **application d'IA complète, déployée, documentée** — construite sur des données réelles. C'est exactement ce qu'un recruteur demande à voir.

“ Deux règles simples dès maintenant : tout le travail vit dans un **dépôt Git par équipe**, et chaque séance de TP se termine par **un commit qui fonctionne**. ”

La commande (scénario)

Vous êtes l'équipe data/IA de **NutriScope**, une jeune société lilloise qui veut lancer un service d'aide au choix alimentaire : on scanne un produit en magasin, l'application explique sa qualité nutritionnelle et propose des alternatives plus saines.

La direction — jouée par vos formateurs — vous confie la totalité du chantier :

- comprendre le besoin, cadrer le projet et chiffrer ce qu'il rapporte ;
- construire la base de données produits et la fiabiliser ;
- développer les briques d'intelligence : scores, recommandations, lecture d'images, assistant conversationnel ;
- mettre le tout en production et le maintenir.

“ Le scénario est fictif, **les données et les problèmes sont réels**. Vous rencontrerez de la donnée sale, des choix d'architecture discutables, des arbitrages coût/qualité : c'est voulu. ”

Les données : Open Food Facts

Base collaborative mondiale de produits alimentaires, d'origine française, en licence ouverte. C'est notre matière première pour tout le projet.

Ce qu'on y trouve	Détail
Fiches produits	plusieurs millions de références : ingrédients, nutriments, Nutri-Score, catégories, marques, magasins
Exports complets	régénérés chaque nuit — CSV, JSONL et Parquet (recommandé, se travaille très bien avec DuckDB)
Photos des produits	l'intégralité des images est publiée sur un bucket S3 public (programme AWS Open Data), téléchargeable sans compte
API JSON	lecture fiche par fiche, utile pour la future application

Point d'entrée unique : world.openfoodfacts.org/data

“ **Licences à respecter** : ODbL pour la base, CC-BY-SA pour les images. Concrètement : on crédite Open Food Facts dans chaque livrable et on repartage dans les mêmes conditions. Ce n'est pas un détail, c'est une exigence du projet — et un point du module conformité. ”

Ce que vous allez construire

Le produit final assemble des briques que vous développerez au fil des modules :

Socle data — une base propre (PostgreSQL puis fichiers Parquet), alimentée par un pipeline de nettoyage reproductible, avec une analyse exploratoire documentée et des tableaux de bord lisibles par un non-technicien.

Modèles — prédiction du Nutri-Score d'un produit à partir de ses caractéristiques, segmentation du catalogue, moteur de substitution (« proposer mieux »), et un classifieur d'images entraîné avec Keras sur les photos du bucket.

Assistant — un chatbot RAG qui répond aux questions des utilisateurs en s'appuyant sur le catalogue et sur des sources publiques de référence en nutrition ; en option, des agents pour les tâches répétitives.

Industrialisation — une API FastAPI qui expose les modèles, une petite application de démonstration, le tout conteneurisé, déployé sur le cloud avec CI/CD et supervisé.

Conformité — registre RGPD, analyse de biais, positionnement AI Act, accessibilité RGAA. Livré **en même temps** que le reste, pas après.

Organisation

Équipes de 2 à 3. Les équipes sont constituées demain et ne bougent plus — sauf cas de force majeure, comme en entreprise.

Le rythme est celui du planning de la promotion. Les séances marquées « TP PROJET FIL ROUGE » sont encadrées par le formateur du module en cours : c'est lui qui fixe l'objectif de la séance et valide l'incrément. Entre deux séances, rien n'interdit d'avancer.

Chaque phase se conclut par un **jalon livré** : un tag Git, un document court, et cinq minutes de démonstration au groupe. On montre ce qui tourne, pas des slides.

Outils imposés : Python 3.12, Git (dépôt d'équipe), DuckDB/PostgreSQL, scikit-learn, TensorFlow/Keras, FastAPI, Docker. Le reste — éditeur, librairies complémentaires — est libre, à condition de le justifier dans le README.

Langue : code et noms de variables en anglais, documentation et soutenances en français.

Calendrier — phases 1 et 2 (été → septembre)

Séances	Phase	Ce qu'on attend
29/07 (A. Abarji)	P1 — Prise en main	premier contact avec l'export Open Food Facts en pandas, mise en place du dépôt Git
25/08, 27/08 (A. Abarji)	P1 (suite)	exploration approfondie, choix du périmètre produit France, qualité des données constatée
01/09 (A. Abarji)	P1 — fin	base SQL modélisée et chargée (produits, catégories, nutriments) — jalón J1
07/09 → 15/09 (G. Grenade, S. Zaghdoudi)	P2 — Cadrage	note de cadrage NutriScope : besoin, parties prenantes, KPI, budget/ROI, risques
16/09 – 17/09 (S. Zaghdoudi)	P2 — fin	backlog priorisé et plan projet (module 9.1) — jalón J2

“ La phase 2 se traite comme une vraie mission de cadrage : entretiens avec le « client » (vos formateurs), livrable écrit court, soutenu à l'oral. ”

Calendrier — phases 3 et 4 (septembre → début novembre)

Séances	Phase	Ce qu'on attend
24/09, 30/09 (C. Ringot)	P3 — Data engineering	pipeline de nettoyage reproductible ; traitement des valeurs manquantes et aberrantes
07/10 (S. Zaghdoudi)	P3 (suite)	EDA visuelle et storytelling : l'état du marché alimentaire raconté en graphiques
08/10, 14/10 (C. Ringot)	P3 — fin	bascule Parquet + DuckDB, données prêtes pour le ML — jalón J3
20/10, 22/10 (C. Ringot)	P4 — Machine learning	prédiction du Nutri-Score (classification), premières régressions sur les nutriments
28/10 (B. Quinet)	P4 (suite)	segmentation du catalogue, moteur de substitution v1
04/11 (B. Quinet)	P4 — fin	classifieur d'images Keras entraîné sur un échantillon du bucket S3 — jalón J4

“ Chaque modèle est livré avec ses métriques, sa validation croisée et deux paragraphes honnêtes sur ses limites. Un modèle sans évaluation sérieuse ne passe pas le jalón. ”

Calendrier — phases 5 et 6 (novembre → décembre)

Séances	Phase	Ce qu'on attend
09/11 (B. Quinet)	P5 — IA générative	choix d'architecture de l'assistant, premiers appels LLM outillés
16/11, 19/11 (A. Abarji)	P5 (suite)	RAG opérationnel sur le catalogue et un corpus nutrition public (indexation, retrieval, évaluation)
25/11, 30/11 (B. Quinet)	P5 — fin	assistant NutriScope v1 démontrable, garde-fous et tests — jalón J5
02/12 (B. Quinet)	P6 — Industrialisation	API FastAPI exposant score + substitution + assistant
09/12 (B. Quinet)	P6 (suite)	application de démonstration branchée sur l'API
15/12, 21/12 (C. Ringot)	P6 (suite)	conteneurisation Docker, pipeline CI/CD
23/12 (I. Abadi)	P6 (suite)	déploiement cloud effectif
28/12 – 29/12 (autonomie)	P6 — fin	stabilisation en autonomie, revue de code croisée — jalón J6

Calendrier — phase 7, stage et dernière ligne droite

Séances	Phase	Ce qu'on attend
05/01 (A. Abarji)	P7 — Consolidation	monitoring des modèles et de l'API
06/01 – 07/01 (A. Abarji)	P7 (suite)	analyse de biais et d'explicabilité ; dossier conformité (RGPD, AI Act, RGAA, licences)
11/01 → 14/01 (A. Abarji)	P7 — fin	module 9.2 : intégration finale, documentation complète, soutenance blanche — jalón J7
18/01 → 27/02	Stage	le projet est en pause ; profitez du terrain pour confronter vos choix à la réalité
02/03 → 04/03 (I. Abadi, distanciel)	Finitions	retours d'expérience de stage intégrés, derniers correctifs, répétition
08/03 (I. Abadi, distanciel)	Préparation	dossiers de certification bouclés
09/03 → 11/03	Certification	soutenances devant le jury

Jalons et livrables

Jalon	Date	Livrable principal
J1	01/09	dépôt Git structuré + base chargée + note d'exploration (3-4 pages)
J2	17/09	note de cadrage + backlog + plan projet
J3	14/10	pipeline data documenté + rapport EDA + tableaux de bord
J4	04/11	modèles ML et DL évalués (notebooks propres + métriques)
J5	30/11	assistant RAG démontrable + rapport d'évaluation
J6	29/12	application déployée (URL), CI/CD, images Docker
J7	14/01	produit final + documentation + soutenance blanche

“Chaque jalon se démontre : cinq minutes devant le groupe, sur ce qui tourne réellement.”

Comment lire ce cahier

Chaque page qui suit est **le sujet d'une séance TP fil rouge** : ce qu'on fait, dans quel ordre, et ce qui doit être committé avant de partir. Le formateur du jour l'adapte librement — c'est un socle, pas un carcan.

- **½ journée** : le module du matin fournit la technique, le TP de l'après-midi l'applique à NutriScope. **Journée** : intégration et production.
- Tout finit dans le dépôt Git d'équipe ; le carnet `docs/journal.md` prend trois lignes par séance (fait / décidé / bloqué).
- Les règles transverses — IA générative, licences, conformité — sont définies dans le sujet principal et ne sont pas répétées à chaque page.

“ Numérotation : TP 1 à TP 31. Les blocs de plusieurs jours (autonomie de décembre, intégration 9.2, distanciel de mars) comptent pour un seul sujet. ”

TP 1 — Démarrage · mer 29/07 (A. Abarji, journée)

Objectif : toucher les données, monter l'outillage. Rien de plus, mais tout doit tourner.

1. Constitution des équipes (2-3) et création du dépôt Git — arborescence imposée : `data/`, `notebooks/`, `src/`, `docs/`.
2. **Aller chercher les données vous-mêmes** sur `world.openfoodfacts.org/data` : explorer les formats proposés, choisir celui qui tient sur votre machine, le télécharger et documenter votre choix dans `docs/journal.md` (format retenu, taille, date de l'export, pourquoi celui-là).
3. Premier notebook pandas : chargement, dimensions, types, mémoire, taux de remplissage par colonne.
4. Cinq questions à résoudre en équipe : combien de produits vendus en France ? quelle part a un Nutri-Score renseigné ? les dix marques les plus présentes ? le taux de manquants sur les nutriments clés (`energy_100g`, `sugars_100g`, `salt_100g`) ? qu'est-ce qui vous semble le plus « sale » dans ces données ?
5. Restitution : dix minutes par équipe en fin de journée — dont la stratégie de téléchargement retenue, qui diffère sûrement d'une équipe à l'autre.

À committer : dépôt initialisé + notebook + réponses dans `docs/journal.md`. **Les données brutes restent hors du dépôt** — seul le script ou la commande qui les récupère est versionné.

“ Le dump complet fait plusieurs gigaoctets : filtrer à la source (colonnes, pays) fait partie de l'exercice. Prévoir du réseau et de la patience — c'est la première vraie contrainte du projet. ”

TP 2 — Profiling & périmètre · lun 25/08 (A. Abarji, journée)

Objectif : connaître le jeu de données à fond et décider sur quoi porte NutriScope.

1. Profiling systématique des données récupérées au TP 1 : distributions, cardinalités, doublons de codes-barres, incohérences d'unités, valeurs impossibles (sucres > 100 g/100 g, énergies nulles...).
2. Inventaire des colonnes : lesquelles servent le produit (score, substitution, images, assistant), lesquelles sont du bruit. S'appuyer sur `data-fields.txt` d'Open Food Facts.
3. Décision de périmètre en équipe : rayons couverts au lancement (5 à 8 catégories), colonnes conservées, seuil de complétude minimal par produit.
4. Rédaction de `docs/perimetre.md` : périmètre retenu, critères, et surtout **ce qu'on écarte et pourquoi**.
5. Tour des équipes en fin de journée : chaque périmètre est challengé par une autre équipe.

À committer : notebook de profiling + `docs/perimetre.md` argumenté.

“ Un périmètre trop large en août se paie en janvier. Le formateur joue le client : il pousse à couper. ”

TP 3 — Le dépôt au propre · jeu 27/08 (A. Abarji, ½ j après « Git »)

Objectif : appliquer le module Git du matin pour que le dépôt tienne huit mois à plusieurs.

1. Mise en place du flux : branche `main` protégée, branche `dev`, branches de fonctionnalité `feat/...`.
2. Conventions d'équipe écrites dans `CONTRIBUTING.md` : format des messages de commit, taille des PR, qui relit quoi.
3. `.gitignore` sérieux : données brutes, environnements, notebooks de brouillon ; les données restent hors du dépôt, seuls les scripts qui les produisent sont versionnés.
4. Exercice : chaque membre ouvre une pull request réelle (reprise du notebook du TP 2 en fonctions dans `src/`), relue et fusionnée par un coéquipier.
5. Revue croisée express : chaque équipe audite le dépôt d'une autre pendant 15 minutes et dépose ses remarques en issues.

À committer : dépôt restructuré, `CONTRIBUTING.md`, premières PR fusionnées, issues ouvertes.

TP 4 — La base NutriScope · mar 01/09 (A. Abarji, ½ j après « SQL »)

Objectif : modéliser et charger la base relationnelle qui portera tout le projet. Jalon J1.

1. Modélisation : tables `produits`, `marques`, `categories` (avec table de liaison — `categories_tags` est multi-valué), `nutriments`. Schéma discuté avec le formateur avant d'écrire une ligne.
2. Script de chargement **rejouable et idempotent** depuis l'extrait nettoyé du périmètre (TP 2) : on doit pouvoir détruire et recharger la base en une commande.
3. Contraintes et clés : unicité du code-barres, intégrité référentielle, types corrects sur les nutriments.
4. Cinq requêtes de contrôle versionnées : volumétrie par table, produits sans catégorie, top marques, complétude Nutri-Score par rayon, doublons restants.
5. Comparaison des schémas entre équipes en clôture : dix minutes, différences argumentées.

À committer : DDL + script de chargement + requêtes de contrôle + tag `v0.1`. **Jalon J1 atteint.**

TP 5 — Parties prenantes & personas · lun 07/09 (G. Grenade, ½ j après 2.2)

Objectif : savoir pour qui on construit NutriScope et qui décide quoi.

1. Carte des parties prenantes du projet : direction, marketing, équipe data, utilisateurs finaux, Open Food Facts (fournisseur de données), délégué à la protection des données. Pouvoir/intérêt pour chacune — le cadre réglementaire (RGPD, AI Act) est une contrainte, pas une partie prenante.
2. Trois personas utilisateurs, réalistes et différenciés : par exemple parent pressé en hypermarché, personne diabétique, étudiant petit budget. Objectifs, freins, situation d'usage.
3. Trame d'entretien semi-directif (10 questions) pour valider les besoins.
4. Entretien réel : 15 minutes par équipe avec « la direction » (le formateur), enregistré dans un compte-rendu.
5. Mise à jour des personas d'après l'entretien : ce qui a été confirmé, infirmé, découvert.

À committer : docs/cadrage/parties_prenantes.md, personas, trame + compte-rendu d'entretien.

TP 6 — Opportunité & concurrence · mer 09/09 (G. Grenade, ½ j après 2.3)

Objectif : situer NutriScope sur son marché et formuler sa proposition de valeur.

1. Benchmark de l'existant : Yuka, l'application Open Food Facts, myLabel, ScanUp... Pour chacun : fonctionnalités, modèle économique, points faibles visibles.
2. Matrice comparative (fonctionnalités x acteurs) : où sont les cases vides ?
3. Analyse rapide type SWOT de NutriScope au regard du benchmark.
4. Formulation de la proposition de valeur en une phrase testable, puis en un paragraphe : qu'est-ce qu'on fait mieux ou différemment, pour qui.
5. Pitch de 3 minutes par équipe devant le groupe ; vote consultatif sur la proposition la plus convaincante.

À committer : docs/cadrage/benchmark.md (2 pages max) + proposition de valeur.

“ Interdiction d'écrire « grâce à l'IA » dans la proposition de valeur : dire ce que ça change pour l'utilisateur. ”

TP 7 — KPI, coûts & ROI · mar 15/09 (S. Zaghdoudi, ½ j après 2.5)

Objectif : chiffrer le projet comme le ferait une direction financière.

1. Définir 5 à 7 KPI produit mesurables : scans/jour, taux d'activation, rétention à 30 jours, taux de substitution acceptée, coût par requête assistant...
2. Hypothèses de coûts sur 12 mois : hébergement (API + base + front), tokens LLM (estimer un volume de conversations), stockage images, temps homme.
3. Hypothèses de revenus ou d'économies selon le modèle choisi au TP 6 (freemium, B2B, marque blanche...).
4. Calcul de ROI sur trois scénarios : pessimiste, central, optimiste — dans un tableur versionné avec les hypothèses apparentes.
5. Identifier les 3 hypothèses les plus fragiles et comment les vérifier tôt.

À committer : docs/cadrage/kpi_roi.xlsx (ou .ods) + note d'hypothèses.

TP 8 — Note de cadrage & backlog · jeu 17/09 (S. Zaghdoudi, ½ j après 9.1)

Objectif : assembler la note de cadrage et outiller le pilotage. Jalon J2.

1. Assemblage de la note de cadrage (6-8 pages) : contexte, besoin, parties prenantes, périmètre, KPI/ROI, risques avec plans de mitigation, macro-planning aligné sur les jalons J3 à J7.
2. Backlog produit : user stories rédigées (« En tant que..., je veux..., afin de... »), priorisées MoSCoW, estimées grossièrement.
3. Mise en place de l'outil de suivi (issues GitHub/GitLab avec labels, ou tableau kanban) — celui qu'on tiendra vraiment.
4. Rituels d'équipe décidés et écrits : point de 10 min en début de chaque TP, revue à chaque jalon.
5. Présentation de 10 minutes par équipe : la direction (formateur) valide ou renvoie en correction.

À committer : note de cadrage + backlog outillé + `docs/cadrage/planning.md` + tag `v0.2`. **Jalon J2 atteint.**

“ La note se lit sans jargon : elle doit convaincre un investisseur qui n'a jamais vu une ligne de Python. Pièce maîtresse pour BC01. ”

TP 9 — Nettoyage industrialisé · jeu 24/09 (C. Ringot, ½ j après 3.2)

Objectif : transformer les recettes de nettoyage du matin en module testé et rejouable.

1. Créer `src/cleaning.py` : chaque règle de nettoyage devient une fonction pure documentée — normalisation des unités, bornage des nutriments (0–100 g/100 g, énergies plausibles), déduplication des codes-barres, traitement des catégories vides.
2. Stratégie de valeurs manquantes **par colonne et par usage** (supprimer, imputer, garder avec drapeau) — décision écrite, pas implicite.
3. Tests pytest sur chaque règle, y compris les cas tordus relevés au TP 2.
4. Rapport avant/après généré par script : lignes touchées par règle, volumétrie finale.
5. Brancher le nettoyage sur le chargement de la base du TP 4 (la base ne reçoit plus que du propre).

À committer : `src/cleaning.py` + tests verts + `docs/data/rapport_nettoyage.md`.

TP 10 — Analyse exploratoire · mer 30/09 (C. Ringot, ½ j après 3.3)

Objectif : produire l'EDA de référence du projet, celle qu'on citera jusqu'en mars.

1. Distributions des nutriments clés par rayon : que mange-t-on réellement dans le périmètre choisi ?
2. Complétude et qualité par rayon et par marque : où les données sont-elles fiables, où faudra-t-il être prudent ?
3. Corrélations entre nutriments et Nutri-Score : premiers signaux pour le futur modèle (TP 14) — sans conclure trop vite.
4. Analyse des valeurs extrêmes restantes : vraies (boissons énergisantes) ou résiduelles (erreurs de saisie passées entre les mailles) ?
5. Le tout dans un notebook **narratif** : chaque graphique a un titre-message et trois lignes d'interprétation.

À committer : `notebooks/eda_reference.ipynb` + synthèse d'une page dans `docs/data/`.

TP 11 — Datastorytelling · mar 07/10 (S. Zaghdoudi, ½ j après 3.4)

Objectif : raconter l'état du marché alimentaire à la direction, en images.

1. Sélectionner dans l'EDA les 6 à 8 messages qui comptent pour NutriScope (exemples : « 40 % des céréales du périmètre sont notées D ou E », « la complétude s'effondre sur les marques distributeurs »).
2. Un graphique par message, retravaillé : type adapté, titre qui affirme, axes lisibles, une seule idée par visuel — les règles vues le matin.
3. Assembler un mini-deck « État du marché » de 8 slides maximum, charte libre mais cohérente.
4. Présentation de 5 minutes chrono par équipe, questions du groupe.
5. Les graphiques retenus intègrent le futur tableau de bord (phase 6) : les exporter proprement en fonctions réutilisables.

À committer : mini-deck + `src/viz.py` avec les fonctions de tracé.

TP 12 — Parquet & DuckDB · mer 08/10 (C. Ringot, ½ j après 3.5)

Objectif : basculer le stockage analytique et mesurer ce qu'on y gagne.

1. Convertir les données nettoyées en Parquet, partitionnées par rayon ; choisir et justifier les types.
2. Rejouer les cinq requêtes de contrôle du TP 4 en DuckDB directement sur les Parquet.
3. Benchmark honnête : temps et mémoire CSV vs base relationnelle vs Parquet/DuckDB sur trois requêtes types.
4. Clarifier l'architecture cible dans `docs/architecture.md` : la base relationnelle sert l'application, les Parquet servent l'analyse et le ML — qui lit quoi.
5. Préparer le Parquet « features » de la phase 4 : colonnes retenues, types, encodages. Le figeage des jeux d'entraînement et de test se fera au TP 13, dans le pipeline.

À committer : scripts de conversion + benchmark chiffré + `docs/architecture.md` mis à jour.

TP 13 — Pipeline bout-en-bout · mar 14/10 (C. Ringot, journée)

Objectif : une commande unique du CSV brut aux données prêtes pour le ML. Jalon J3.

1. Assembler tous les morceaux (chargement, nettoyage, base, Parquet, exports ML) en un pipeline orchestré :
`Makefile` ou CLI `python -m nutriscope.pipeline`.
 2. Chaque étape est relançable seule ; les sorties sont datées et reproductibles (graines fixées).
 3. Figer les jeux d'entraînement/test de la phase 4 : split documenté, stratifié sur le Nutri-Score, gelé dans `data/exports/` (hors Git, mais régénérable).
 4. Test grandeur nature : chaque équipe clone le dépôt d'une autre sur une machine vierge et déroule son README. Ce qui casse est corrigé dans la journée.
 5. Rédaction de `docs/data/pipeline.md` : schéma du flux, commandes, durées.
- À committer** : pipeline fonctionnel + doc + tag `v0.3`. **Jalon J3 atteint.**

TP 14 — Prédire le Nutri-Score · lun 20/10 (C. Ringot, ½ j après 4.3)

Objectif : première brique ML sérieuse — classification A→E depuis les caractéristiques produit.

1. Baselines d'abord : classe majoritaire, puis arbre de décision simple. Ce sont elles qu'il faudra battre — les noter.
2. Modèles du matin appliqués : régression logistique multinomiale, random forest ; pipeline scikit-learn complet (préprocesseur inclus) pour éviter toute fuite de données.
3. Validation croisée 5 plis stratifiée ; métriques adaptées au déséquilibre (F1 macro, matrice de confusion normalisée).
4. Lecture des variables importantes : est-ce cohérent avec la définition officielle du Nutri-Score ? Si le modèle triche (fuite via une colonne dérivée du score), le dire et corriger.
5. Trois erreurs types analysées à la main, produits réels à l'appui.

À committer : `notebooks/ml_nutriscore.ipynb` + tableau comparatif baseline/modèles.

TP 15 — Méthodes d'ensemble · mer 22/10 (C. Ringot, ½ j après 4.4)

Objectif : pousser la performance proprement et savoir dire ce que le modèle rate.

1. Gradient boosting sur le même problème ; recherche d'hyperparamètres raisonnée (pas de grid search aveugle de trois heures).
2. Comparaison à conditions égales avec les modèles du TP 14 : mêmes plis, mêmes métriques, tableau unique.
3. Analyse des confusions restantes : où le modèle hésite-t-il (B/C ? D/E ?) et pourquoi — retour aux données.
4. Choix du modèle retenu pour l'application, avec ses critères (performance, latence, simplicité de maintenance) tracés dans une ADR (`docs/decisions/`).
5. Rédiger la note honnête « ce que le modèle rate » : limites, biais possibles par rayon, cas où ne pas lui faire confiance.

À committer : notebook comparatif + ADR + `docs/ml/limites_nutriscore.md`.

TP 16 — Segmentation & substitution · mer 28/10 (B. Quinet, journée)

Objectif : le cœur de la promesse NutriScope — « voici un produit comparable, mieux noté ».

1. Matin — clustering du catalogue : normalisation des nutriments, K-Means (choix de k justifié : coude + silhouette), interprétation métier de chaque groupe, nommage.
2. Après-midi — moteur de substitution v1 : pour un produit donné, candidats de la même sous-catégorie, filtrés par distance nutritionnelle, classés par Nutri-Score puis par proximité.
3. Règles métier de garde : pas de substitution inter-rayons absurde, respect des labels le cas échéant (bio → proposer bio en priorité).
4. Fonction propre `src/substitution.py` avec ses tests, callable par la future API.
5. Validation à la main sur 20 produits réels du périmètre : chaque équipe note la pertinence des 3 premières suggestions ; les échecs sont documentés.

À committer : notebook clustering + `src/substitution.py` + rapport des 20 cas.

TP 17 — Classifieur d'images · mer 04/11 (B. Quinet, ½ j après 5.1)

Objectif : la brique deep learning — reconnaître la catégorie d'un produit sur sa photo. Jalon J4.

1. Prendre en main le jeu d'images fourni (2 000–5 000 photos du bucket Open Food Facts, 5-8 catégories équilibrées, préparé en amont par le formateur — on n'y passe pas la séance).
2. CNN simple entraîné de zéro d'abord : architecture minimale, quelques époques — fixer la référence.
3. Transfer learning ensuite (MobileNet ou équivalent vu le matin) avec augmentation de données ; comparer.
4. Évaluation : accuracy par classe, matrice de confusion, et surtout une planche des erreurs les plus parlantes (les photos que le modèle confond, affichées).
5. Deux pistes d'amélioration argumentées en conclusion — celles qu'on tenterait avec plus de temps.

À committer : `notebooks/dl_images.ipynb` complet + modèle sauvegardé + tag `v0.4`, qui clôt l'ensemble des travaux de modélisation des TP 14 à 17 (Nutri-Score, ensembles, substitution, images). **Jalon J4 atteint.**

TP 18 — Architecture de l'assistant · lun 09/11 (B. Quinet, ½ j après 6.1)

Objectif : poser l'assistant NutriScope et réussir le premier appel outillé.

1. Cadrer l'assistant : ce qu'il sait faire (répondre sur un produit, comparer, expliquer un score, recommander) et ce qu'il refuse (conseil médical, hors alimentation).
2. Choix du modèle et du mode d'accès (API ou modèle local), critères coût/latence/confidentialité tracés en ADR.
3. Premier outil branché : fonction calling « chercher_produit » qui interroge la base NutriScope et rend une fiche structurée.
4. Dialogue de démonstration : l'utilisateur demande un produit, le LLM appelle l'outil, la réponse cite les données réelles.
5. Schéma d'architecture de la cible (LLM, outils, futur RAG, API) dans `docs/architecture.md`.

À committer : prototype d'appel outillé + ADR + schéma à jour.

TP 19 — Construire le RAG · lun 16/11 (A. Abarji, journée)

Objectif : l'assistant répond en s'appuyant sur un corpus, avec des sources.

1. Constituer le corpus : fiches produits du périmètre (générées depuis la base) + documents nutrition publics de référence (repères PNNS, pages officielles Nutri-Score) — provenance et licence notées pour chaque source.
2. Découpage : stratégie de chunking adaptée (une fiche produit = un chunk ; documents longs découpés avec chevauchement), justifiée.
3. Vectorisation et indexation dans la base vectorielle vue en cours (Qdrant ou Chroma), pipeline d'ingestion rejouable.
4. Chaîne RAG complète : question → retrieval top-k → génération avec citations des sources dans la réponse.
5. Premiers essais libres en fin de journée : chaque équipe pose 5 questions à l'assistant d'une autre et note ce qui cloche.

À committer : pipeline d'ingestion + chaîne RAG + `docs/ia/corpus.md` (sources et licences).

TP 20 — Évaluer le RAG · jeu 19/11 (A. Abarji, journée)

Objectif : passer de « ça a l'air de marcher » à une qualité mesurée, puis améliorée.

1. Construire un jeu de 30 questions de référence avec réponses attendues et sources correctes : questions factuelles, comparatives, et 5 questions pièges hors périmètre.
2. Mesurer le retrieval : le bon chunk est-il dans le top-k (hit rate, MRR) ? Où échoue-t-il ?
3. Mesurer les réponses avec une grille simple (exactitude, fidélité aux sources, refus corrects des pièges), remplie en double aveugle entre équipes.
4. Deux itérations d'amélioration au choix : taille de chunks, k, reformulation de requête, reranking — mesurées à chaque fois sur les mêmes 30 questions.
5. Tableau avant/après en synthèse : qu'est-ce qui a réellement fait progresser la qualité ?

À committer : jeu d'évaluation versionné + résultats des itérations + synthèse.

TP 21 — Orchestration & outils · mer 25/11 (B. Quinet, ½ j après 6.4)

Objectif : un assistant multi-tours qui combine RAG, base produits et substitution ML.

1. Reprendre la chaîne dans le framework vu le matin (LangChain) : gestion de la conversation, mémoire courte.
2. Déclarer les outils : `chercher_produit` (TP 18), `substituer` (TP 16), `repondre_source` (RAG des TP 19-20).
L'assistant choisit le bon outil selon la question.
3. Scénarios multi-tours à faire passer : « ce produit est-il bon ? → pourquoi ? → propose mieux → et en bio ? ».
4. Journalisation : chaque échange trace outil appelé, latence, tokens — on en aura besoin en phase 7.
5. Revue croisée : 10 minutes de conversation libre avec l'assistant d'une autre équipe, remarques en issues.

À committer : assistant orchestré + scénarios rejouables + journalisation active.

TP 22 — Garde-fous & gel de la v1 · lun 30/11 (B. Quinet, ½ j après 6.5)

Objectif : rendre l'assistant présentable à des utilisateurs réels. Jalon J5.

1. Prompt système définitif : rôle, ton, périmètre, consignes de refus (conseil médical, allergies engageantes, hors alimentation), obligation de citer ses sources.
2. Tests adverses croisés : chaque équipe prépare 10 attaques (injection, questions médicales déguisées, demandes hors sujet insistantes) et les lance sur l'assistant d'une autre.
3. Correction des failles trouvées ; les attaques deviennent des tests automatisés qui rejoignent la suite.
4. Décision tracée sur le fine-tuning vu le matin : utile pour NutriScope ou non ? ADR courte, dans les deux cas.
5. Gel de l'assistant v1 : plus de changement de comportement avant la phase 6.

À committer : prompt système versionné + rapport des tests croisés + ADR + tag `v0.5`. **Jalon J5 atteint.**

TP 23 — L'API NutriScope · mer 02/12 (B. Quinet, ½ j après 7.1)

Objectif : exposer les briques en une API propre, contractuelle, testée.

1. FastAPI, quatre routes : `GET /produits/{code}`, `GET /produits/{code}/score`, `GET /produits/{code}/substituts`, `POST /assistant`.
2. Schémas Pydantic en entrée et en sortie ; gestion d'erreurs propre (produit inconnu, code invalide, assistant indisponible).
3. La doc Swagger générée fait foi : descriptions, exemples, codes de retour — quelqu'un d'extérieur doit pouvoir consommer l'API sans nous appeler.
4. Tests d'API avec httpx/pytest : cas nominaux et cas d'erreur pour chaque route.
5. Mesure de latence par route ; noter celles qui poseront problème en production (l'assistant, évidemment) et les options.

À committer : `src/api/` + tests verts + capture de la doc dans `docs/api.md`.

TP 24 — L'application de démonstration · mer 09/12 (B. Quinet, journée)

Objectif : ce que verront le jury et les utilisateurs — une appli branchée sur l'API.

1. Choix assumé de la techno de front (Streamlit pour aller vite, ou front léger si l'équipe est à l'aise) — ADR de deux paragraphes.
2. Parcours à couvrir : rechercher un produit (nom ou code-barres), afficher sa fiche et son score expliqué, voir les alternatives proposées, discuter avec l'assistant.
3. L'appli ne parle **qu'à l'API** — aucune requête directe à la base ni au modèle. C'est la règle d'architecture, elle sera vérifiée.
4. États soignés : chargement, produit introuvable, assistant lent — pas d'écran blanc.
5. Démo croisée en fin de journée : une personne extérieure à l'équipe déroule le parcours sans aide.

À committer : appli fonctionnelle + guide d'utilisation d'une page.

TP 25 — Conteneurisation · mar 15/12 (C. Ringot, ½ j après 7.3)

Objectif : NutriScope tourne sur n'importe quelle machine en une commande.

1. Dockerfile de l'API : image légère, multi-étages si utile, dépendances figées, utilisateur non-root.
2. Dockerfile de l'appli ; `docker-compose.yml` orchestrant API + appli + base + base vectorielle.
3. Configuration par variables d'environnement (fichier `.env.example` complet, jamais de secret en dur) ; volumes pour les données persistantes.
4. Épreuve de vérité : `docker compose up` sur une machine qui n'a jamais vu le projet — chaque équipe teste celui d'une autre.
5. Consigner tailles d'images et temps de build ; une optimisation appliquée si le résultat est déraisonnable.

À committer : Dockerfiles + compose + `.env.example` + `docs/deploiement.md` (début).

TP 26 — Chaîne CI/CD · lun 21/12 (C. Ringot, journée)

Objectif : plus aucune mise en production à la main.

1. Pipeline CI sur le dépôt : lint (ruff), tests unitaires et tests d'API à chaque push ; le pipeline rouge bloque la fusion sur `main`.
2. Build et publication des images Docker taguées (SHA + version) vers un registre.
3. Étape de déploiement automatisé, déclenchée sur tag, vers un environnement de **pré-production monté dans la séance** (le compose du TP 25 sur une machine ou un conteneur dédié). Le TP 27 rebranchera cette même étape sur la production.
4. Gestion des secrets côté CI (variables protégées) — vérification qu'aucun secret ne traîne dans l'historique Git.
5. Preuve par l'exemple : une correction triviale part du commit à l'image publiée puis déployée en pré-production, sans intervention manuelle.

À committer : fichiers de pipeline + badge de statut dans le README + journal d'un déploiement réussi en pré-production.

TP 27 — Mise en ligne · mer 23/12 (I. Abadi, ½ j après 7.5)

Objectif : NutriScope accessible depuis n'importe quel navigateur.

1. Déploiement sur l'environnement cloud vu le matin (VPS ou PaaS selon les comptes disponibles — fourni par le centre), puis **rebranchement du pipeline du TP 26** sur cette cible : le déploiement en production passe par la CI, jamais à la main.
2. Reverse proxy et HTTPS ; nom de domaine ou URL de service propre.
3. Checklist de mise en ligne déroulée : santé des conteneurs, migrations jouées, variables d'environnement de prod, sauvegardes de la base planifiées, journaux accessibles.
4. Test de bout en bout depuis l'extérieur du réseau du centre (4G) : recherche, score, substituts, assistant.
5. L'URL est communiquée au groupe et notée dans le README — elle ne changera plus.

À committer : configuration de déploiement + `docs/déploiement.md` finalisé + URL publique.

TP 28 — Autonomie · lun 28 & mar 29/12 (sans encadrant)

Objectif : stabiliser la v1.0 en conditions réelles d'équipe. Jalon J6.

1. Jour 1 matin — backlog de stabilisation : trier tout ce qui reste (bugs, dette, doc), estimer, décider ce qui entre dans la v1.0 et ce qui est assumé comme reporté.
2. Jour 1 — corrections en autonomie ; la discipline Git et CI reste entière (PR, revues internes, pipeline vert).
3. Jour 2 matin — revue de code croisée inter-équipes : grille fournie (lisibilité, tests, sécurité de surface, doc), une heure par dépôt, remarques en issues.
4. Jour 2 — traitement des remarques bloquantes ; les autres sont étiquetées pour janvier.
5. Rapport d'autonomie d'une page : fait, reporté, décisions prises seuls — et tag `v1.0`.

À committer : correctifs + issues de revue + rapport + tag `v1.0`. **Jalon J6 atteint.**

“ Les formateurs ne sont pas joignables ces deux jours : c'est volontaire. Les blocages non résolus se documentent, ils ne se contournent pas en cachette. ”

TP 29 — Monitoring · lun 05/01 (A. Abarji, journée)

Objectif : savoir en permanence si NutriScope va bien — et le prouver.

1. Healthchecks exposés par l'API (`/health`, `/ready`) et surveillés ; alerte simple en cas de chute (mail ou webhook).
2. Journaux structurés (JSON) centralisés pour l'API et l'assistant : chaque requête trace route, latence, statut ; chaque conversation trace outil, tokens, coût estimé.
3. Tableau de bord d'exploitation : trafic, latences P50/P95 par route, taux d'erreur, coût LLM cumulé — l'outillage vu le matin ou un dashboard maison branché sur les journaux.
4. Suivi de la qualité des modèles dans le temps : **automatiser** le rejeu du jeu d'évaluation RAG (TP 20) et d'un échantillon de prédictions Nutri-Score — tâche planifiée ou job de CI —, avec archivage des résultats pour détecter une dérive. La mise en place se fait aujourd'hui, la lecture des courbes se fera au TP 30.
5. Exercice de panne : le formateur casse quelque chose sur l'environnement de chaque équipe ; on doit le détecter par le monitoring, pas par hasard.

À committer : supervision active + tableau de bord + `docs/exploitation.md`.

TP 30 — Intégration finale & soutenance blanche · lun 11 → jeu 14/01 (A. Abarji, module 9.2)

Objectif : livrer le produit complet et le défendre. Jalon J7.

1. **Lundi — documentation** : README définitif, guide d'installation testé, architecture à jour, dossier conformité relu (RGPD données simulées, AI Act, RGAA, crédits Open Food Facts).
2. **Mardi — parcours de démonstration** : scénario minuté de bout en bout avec données de démo fiables ; plan B hors ligne si le réseau lâche le jour J.
3. **Mercredi — répétition** : soutenance complète à blanc entre équipes, chronométrée (20 min + 10 de questions), questions méchantes encouragées.
4. **Jeudi — soutenance blanche officielle** devant les formateurs, conditions du jury ; retours écrits remis à chaque équipe.
5. Consolidation du dossier de projet : tout ce qui servira aux dossiers de certification par bloc est rangé et pointé depuis un sommaire unique.

À committer : dossier de projet complet + retours de soutenance archivés + **jalons J7**.

TP 31 — Retour de stage & certification · 02-04/03 et 08/03 (I. Abadi, distanciel)

Objectif : transformer huit mois de travail en certification.

1. **02/03** — reprise : traiter les retours écrits de la soutenance blanche un par un (fait / abandonné avec justification) ; intégrer ce que le stage a appris à chacun dans le discours.
2. **03/03 (½ j)** — derniers correctifs techniques uniquement (pas de fonctionnalité nouvelle à J-6) ; gel définitif **v1.1**.
3. **04/03** — répétition générale en visio : passage complet minuté par équipe, questions croisées, réglage des démos en conditions distancielles.
4. **08/03** — dossiers de certification bloc par bloc (BC01 → BC05) : pour chaque compétence, la preuve pointée dans le projet (document, commit, écran) ; relecture croisée puis dépôt.
5. Check matériel et logistique pour les 09-11/03 : qui présente quoi, sur quelle machine, avec quel filet de sécurité.

À livrer : tag **v1.1** + dossiers déposés + supports de soutenance finaux.

31 sujets, 7 jalons, un produit.

Rendez-vous demain 9h — TP n°1 avec Ayoub.

Contact pédagogique : **Ihab Abadi** — UTOPIOS

Centre du Général de Gaulle, 59200 Tourcoing

Données : Open Food Facts (ODbL / CC-BY-SA)